wTAO Audit Report

Prepared by Cyfrin

Version 2.0

Lead Auditors

Immeas

Farouk

May 1, 2026

Contents

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

`wTAO` is a small Solidity contract that merges a WETH9-style native wrapper with a LayerZero v2 `OFT` for omnichain transfers. `deposit` / `receive` mints wTAO 1:1 against native TAO; `withdraw` burns and releases native. Cross-chain transfers burn wTAO on the source chain and mint it on the destination chain, with delivery secured by LayerZero's configured peers and DVNs.

Intended deployment will be on Subtensor EVM (chain ID 964) as the origin holding the native TAO, with a plain OFT representation of wTAO on Solana for the initial cross-chain corridor.

5 Audit Scope

The audit scope was limited to:

```
src/wTAO.sol
scripts/DeploywTAO.s.sol
Makefile
```

6 Executive Summary

Over the course of 1 days, the Cyfrin team conducted an audit on the wTAO code provided by TAO. The findings consist of 3 Low severity issues with the remainder being informational and gas optimizations.

6.1 Centralization Risks

The owner role on the deployed `wTAO` contract holds the full LayerZero OApp admin surface inherited from `OAppCore` / `OFTCore` / `Ownable` (`setPeer`, `setDelegate`, `setEnforcedOptions`, `setMsgInspector`, `setPreCrime`, `transferOwnership`, `renounceOwnership`). Per the deploy SOP the broadcasting EOA becomes the owner directly at

construction (OZ v4.9 `Ownable` parameterless constructor sets `_owner = msg.sender`), so the production owner key is whichever Safe / cold key the team uses to broadcast `script/DeploywTAO.s.sol`. The `OFT_OWNER` env var sets the LayerZero endpoint delegate as a separately scoped role (a strict subset of owner — the owner can replace the delegate at any time via `setDelegate`).

Owner can DoS or censor user cross-chain operations via the inherited OApp admin functions (`setPeer`, `setMsgInspector`, `setEnforcedOptions`) without any restriction at the contract layer. The same set of admin paths produces effects at two points in the message lifecycle:

- *Outbound, before send.* `setMsgInspector` installs a contract that reverts on every send (or on selected payloads). `setEnforcedOptions` set with absurd gas makes the LZ fee quote unaffordable, or with zero gas makes the destination `lzReceive` OOG while the origin burn still succeeds.
- *In-flight, after source-side burn.* `setPeer(srcEid, ...)` mid-flight makes `OAppReceiver._getPeerOrRevert` reject an inbound message that was already authenticated against the prior peer; the same shape applies to mid-flight `setMsgInspector` / `setEnforcedOptions` changes. The user's source-side burn cannot be unwound.

This is inherent to the LayerZero OApp pattern and accepted for OFTs whose admin role is held by a multisig or governance contract; mitigation is operational (timelock or multisig discipline on these admin paths) rather than code-level. Surfaced here so the trust assumption is explicit.

Summary

Project Name	wTAO
Repository	wrapped-tao-smart-contracts
Commit	7fe66659b893...
Fix Commit	ba285373e4ce...
Audit Timeline	Apr 29th - Apr 29th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	3
Informational	3
Gas Optimizations	1
Total Issues	7

Summary of Findings

[L-1] Missing `renounceOwnership` override: a single owner tx permanently bricks all `wTAO` admin functions	Resolved

[L-2] Default `setEnforcedOptions` policy is destination-class-blind and msgType-incomplete; first send to a fresh Solana ATA can fail	Acknowledged
[L-3] Makefile broadcasts deploys with clear-text key passing	Resolved
[I-1] Constructor parameter `owner` is a misleading name	Resolved
[I-2] Consider supporting `SEND_AND_CALL`	Acknowledged
[I-3] Foundry profile drift between production and CI - tests do not exercise the deployed bytecode	Acknowledged
[G-1] Use low-level call to send ETH if return data is unimportant	Resolved

7 Findings

7.1 Low Risk

7.1.1 Missing `renounceOwnership` override: a single owner tx permanently bricks all `wTAO` admin functions

Description: `wTAO` (`src/wTAO.sol:24-58`) does not override `Ownable.renounceOwnership`. The owner can call `renounceOwnership()` and permanently set `_owner = address(0)`, which freezes `setPeer`, `setDelegate`, `setEnforcedOptions`, `setMsgInspector`, and `setPreCrime` at their last state with no on-chain recovery path. Even with a properly-deployed Safe multisig as the broadcaster (so the multisig is the Ownable owner), a faulty Safe transaction proposal that calls `renounceOwnership()` permanently disables governance over the corridor.

Impact: Single accidental call from the owner permanently disables every admin capability on `wTAO`. Cross-chain peer reconfiguration becomes impossible; if a peer is later compromised and the owner needs to call `setPeer(0)` to revoke it, the call reverts. Any defense-in-depth additions (rate limiter setters, pause toggles) that the protocol later wires through Ownable also become permanently inaccessible.

Recommended Mitigation: Override `renounceOwnership` in `wTAO` to revert unconditionally:

```
function renounceOwnership() public pure override {
    revert("renounceOwnership disabled");
}
```

This preserves the ability to `transferOwnership` to a new multisig while removing the foot-gun of permanent `address(0)` ownership.

wTAO: Fixed in commit bc35833

Cyfrin: Verified.

7.1.2 Default `setEnforcedOptions` policy is destination-class-blind and msgType-incomplete; first send to a fresh Solana ATA can fail

Description: script/DeploywTAO.s.sol:21-50 deploys `wTAO` and seeds exactly one enforced-option entry on the new OApp:

```
uint16  internal constant SEND_MSG_TYPE     = 1;       // line 29
uint128 internal constant LZ_RECEIVE_GAS    = 80_000;  // line 31
uint128 internal constant LZ_RECEIVE_VALUE  = 0;       // line 33

EnforcedOptionParam[] memory enforcedOptions = new EnforcedOptionParam[](1);
enforcedOptions[0] = EnforcedOptionParam({
    eid:     dstEid,
    msgType: SEND_MSG_TYPE,
    options: _buildLzReceiveOption(LZ_RECEIVE_GAS, LZ_RECEIVE_VALUE)
});                                                    // lines 43-48
deployed.setEnforcedOptions(enforcedOptions);          // line 49
```

The encoder at script/DeploywTAO.s.sol:71-75 packs `gas` (and optionally `value`) into a type-3 `LZ_RECEIVE` executor option:

```
bytes memory option = value == 0 ? abi.encodePacked(gas) : abi.encodePacked(gas, value);
return abi.encodePacked(
    OPTION_TYPE_3, EXECUTOR_WORKER_ID, uint16(option.length + 1), EXECUTOR_OPTION_TYPE_LZ_RECEIVE,
    ↪  option
);
```

`OAppOptionsType3.combineOptions(_eid, _msgType, _extraOptions)` merges this enforced entry with caller-provided `extraOptions` for every outbound `send`. Because the script does not branch on destination class, the same `(gas=80_000, value=0)` policy applies to EVM and Solana destinations alike — and that produces a delivery-quality issue on Solana corridors where the recipient ATA has not yet been initialized.

On EVM destinations the `value` field of `LZ_RECEIVE` is forwarded as `msg.value` into the destination `lzReceive` call; for plain OFT credits this is correctly zero. On a Solana destination the same field instructs the executor how many lamports to attach to the destination instruction so it can fund SPL Associated Token Account rent if the recipient ATA needs to be created. The rent-exempt minimum for a 165-byte SPL token account is approximately 2,039,280 lamports.

The hazard is conditional on whether the recipient already has the wTAO SPL ATA:

- **Recipient ATA already exists:** `value=0` is fine. No rent is needed for the credit step; the executor delivers `lzReceive` and the SPL is credited normally.
- **Recipient ATA does not yet exist:** the executor cannot fund `createAssociatedTokenAccount`, so `lzReceive` fails on Solana. Origin wTAO is already burned by `_debit` (lib/devtools/packages/oft-evm/contracts/OFTCore.sol:231-253).

In the failure case, the message is **delayed, not lost**. LayerZero stores the verified-but-unexecuted packet on the destination endpoint, and recovery is for the recipient (or any third party) to pre-create the wTAO SPL ATA so the executor's retry of `lzReceive` no longer needs to fund `createAssociatedTokenAccount`; the executor then re-executes the stored packet and the credit lands. Note that the packet's executor options were sealed into the verified payload at source-side `send` time, so calling `setEnforcedOptions` after the fact only changes the policy for *future* sends — it does not retroactively alter the in-flight packet's options. The fix-forward path for the stuck packet is therefore the ATA-creation route, not a source-side rewire.

The Solana scaffold's own solana-oft/layerzero.config.ts:30-46 already defines the canonical `SOLANA_ENFORCED_OPTIONS` (`gas = CU_LIMIT = 200_000`, `value = SPL_TOKEN_ACCOUNT_RENT_VALUE = 2_039_280`). The Foundry script silently overrides those values whenever it is run with a Solana destination EID (per Makefile:63, `make deploy-wtao` only forwards `DST_EID` into `run()`). An operator running `make deploy-wtao DST_EID=40168` (Solana Devnet) without subsequently re-running `lz:oapp:wire` against the Solana config produces a corridor that is fragile for first-time recipients.

Impact: First send to a Solana destination whose recipient ATA is not yet initialized lands in a verified-but-unexecuted state. The credit is recoverable by pre-creating the recipient ATA and retrying the executor; the in-flight packet's options are fixed at send time, so a source-side `setEnforcedOptions` rewire only fixes future sends, not the stuck packet. No funds are permanently lost; the user experiences a delivery delay and a UX-confusing failure.

Recommended Mitigation:

- Parameterize `LZ_RECEIVE_VALUE` per destination class. Either accept an `IS_SOLANA_DST` env flag or, preferably, a per-`dstEid` lookup table that maps known Solana EIDs (e.g., Devnet `40168` and the equivalent mainnet IDs) to `(gas = 200_000, value = SPL_TOKEN_ACCOUNT_RENT_VALUE)` while leaving EVM destinations at `value=0`. This eliminates the first-time-recipient delivery failure on Solana corridors at deploy time.
- Align the Foundry script with the canonical Solana values defined in solana-oft/layerzero.config.ts:30-46 so the two configuration sources cannot disagree, or hard-fail in the script when `DST_EID` matches a known Solana EID and require operators to use the Solana scaffold's `lz:oapp:wire` flow instead.

TAO: Acknowledged. We didn't fix setEnforcedOptions because values are going to be overridden during the deployment process. Also no one will be able to send anything before i initialize anything on Solana until I finish all the steps

7.1.3 Makefile broadcasts deploys with clear-text key passing

Description: Every deploy target in the Makefile passes the production private key as a command-line argument:

- `Makefile:58` → `DEPLOY_BASE_ARGS = --broadcast --private-key $(PRIVATE_KEY) $(VERIFY_ARGS)`
- `Makefile:64` → `forge create ... --private-key $(PRIVATE_KEY) ...`
- `Makefile:65` → `cast send ... --private-key $(PRIVATE_KEY) ...`

Passing private keys as command-line arguments is a well-known anti-pattern. The key value enters multiple unintended surfaces:

- `ps` / `/proc/<pid>/cmdline`: any process on the same machine (CI runner, sidecar container, malware, an over-privileged log shipper) can read full command lines including `--private-key 0x...`.
- **Shell history**: if the operator sets `PRIVATE_KEY=0x...` inline (`PRIVATE_KEY=0x... make deploy-wtao`) the value lands in `~/.bash_history` / `~/.zsh_history`.
- **CI logs**: many CI runners log resolved Make-expanded commands. `--private-key 0x...` leaks into those logs unless the CI explicitly redacts it.
- **Crash dumps** / **strace** / **debugger attaches**: command-line args are part of the process's accessible state.

The Makefile already declares an alternative path at `Makefile:39` — `ADDITIONAL_ARGS_BASE = --account mainnetDeployer --sender 0x00 --with-gas-price $(GAS_PRICE)` — which would use Foundry's keystore (encrypted on disk via `cast wallet import`, decrypted in-process only at signing time, never appearing on the command line). But that branch is dead code: `ADDITIONAL_ARGS_BASE` only flows into the `USE_PRIVATE_KEY=0` branch of `ADDITIONAL_ARGS` (`Makefile:42-48`), which `DEPLOY_BASE_ARGS` and the direct-deploy / `cast send` recipes never consume. So the keystore option is in the file but unreachable from the shipped deploy paths.

Impact: Production private key has multiple unnecessary exposure surfaces (process listing, shell history, CI logs, crash dumps). Any of these surfaces being readable by an attacker (CI compromise, sloppy log retention, multi-tenant deploy host, accidentally-copied terminal session) yields the production owner key — which under the team's deploy SOP is the wTAO Ownable owner with full admin authority over the contract.

Recommended Mitigation: Switch all deploy recipes to Foundry's keystore by default. The operator imports the production key once with `cast wallet import mainnetDeployer --interactive` (encrypted on disk under `~/.foundry/keystores/mainnetDeployer`); deploys then reference it by name with `--account mainnetDeployer`, and Foundry prompts for the passphrase at signing time. Concrete edits:

```
# Replace --private-key everywhere with --account
DEPLOY_BASE_ARGS = --broadcast --account $(ACCOUNT) --sender $(SENDER) $(VERIFY_ARGS)
ACCOUNT ?= mainnetDeployer
SENDER  ?= # set to the keystore account's address explicitly

deploy-wtao:; forge script script/DeploywTAO.s.sol:DeploywTAO --sig "run()" --rpc-url $(RPC_URL)
↪  $(DEPLOY_BASE_ARGS)
deploy-wtao-direct:; forge create src/wTAO.sol:wTAO --broadcast --rpc-url $(RPC_URL) --account
↪  $(ACCOUNT) --sender $(SENDER) --legacy --constructor-args "$(LZ_ENDPOINT)" "$(OFT_OWNER)"
set-enforced-options-direct:; cast send $(WTAO_ADDRESS) "setEnforcedOptions((uint32,uint16,bytes)[])"
↪  "[($(DST_EID),1,$(ENFORCED_OPTION_LZ_RECEIVE))]" --rpc-url $(RPC_URL) --account $(ACCOUNT) --legacy
```

Document the one-time setup in the README:

```
cast wallet import mainnetDeployer --interactive   # paste private key, set passphrase
make deploy-wtao RPC_URL=https://mainnet.example   # forge prompts for passphrase
```

If a `--private-key` fallback is genuinely needed for local testing, gate it behind an explicit opt-in flag (e.g., `USE_PRIVATE_KEY=1`) wired through `DEPLOY_BASE_ARGS` rather than the other way around — making keystore the default and key-in-cleartext the special case.

wTAO: Fixed in commit bc35833

Cyfrin: Verified.

7.2 Informational

7.2.1 Constructor parameter `owner` is a misleading name

Description: Constructor parameter `owner` is a misleading name and the natdoc *"Owner/delegate address used by OFT/OApp configuration"* (`src/wTAO.sol:29`) overstates the role. The parameter is **only** the LayerZero endpoint delegate (forwarded to `OAppCore` and then to `endpoint.setDelegate`); the `Ownable.owner` is set independently to `msg.sender` at construction time (i.e. the broadcaster).

Rename the parameter to `lzDelegate` (or similar), update the natdoc to read *"LayerZero endpoint delegate. Note: the on-wTAO Ownable owner is set to msg.sender at construction time and is a separate role"*, and update `test/DeploywTAO.t.sol:25-28` to assert the same expectation explicitly with a comment explaining the deploy SOP (broadcaster = production owner key).

wTAO: Fixed in commit bc35833

Cyfrin: Verified.

7.2.2 Consider supporting `SEND_AND_CALL`

Description: Consider supporting `SEND_AND_CALL` (compose) by adding a `msgType=2` entry to `setEnforcedOptions` at deploy time. Today the deploy script (`script/DeploywTAO.s.sol:43-49`) sets enforced options only for `msgType=1` (`SEND`). `OFTCore._buildMsgAndOptions` switches to `msgType=2` automatically whenever a user passes a non-empty `composeMsg`; with no `(eid, 2)` enforced entry and an empty `extraOptions` from the user, the destination message ships with zero executor gas. The owner can fix this any time post-deploy via `setEnforcedOptions`.

Use cases compose unlocks for wTAO integrators (all "bridge + execute on destination" in one user-signed flow):

- **Bridge-and-swap** — wrap + bridge + swap on the destination DEX atomically.
- **Bridge-and-stake** / **deposit** — deliver wTAO directly into a destination staking vault, lending market, perpetuals contract, or LP position.
- **Cross-chain treasury** — Safe multisig on one chain ships wTAO + execution payload to a treasury composer on another.
- **Aggregator integration** — drop-in compatibility with LiFi / Socket / Squid bridge-and-execute flows.
- **Bridge-and-pay** — merchant payment routing, invoice settlement, split payouts on the destination chain.

Note: wTAO itself does not implement `IOAppComposer.lzCompose` and is not the intended composer - the integrator's compose-target contract on the destination chain is. wTAO's role in compose is purely as the bridged asset; the composer is downstream. To support compose properly, the deploy script (or a post-deploy `setEnforcedOptions` call) should add a second entry roughly:

```
// gas for destination _credit + dispatch to endpoint.sendCompose
bytes memory lzReceiveOpt   = _buildLzReceiveOption(80_000, 0);
// gas for the composer's lzCompose body - sized per integrator profile (200_000 is a typical floor)
bytes memory lzComposeOpt   = _buildLzComposeOption(200_000, 0);

EnforcedOptionParam[] memory enforcedOptions = new EnforcedOptionParam[](2);
enforcedOptions[0] = EnforcedOptionParam({eid: dstEid, msgType: 1, options: lzReceiveOpt});
enforcedOptions[1] = EnforcedOptionParam({eid: dstEid, msgType: 2, options: bytes.concat(lzReceiveOpt,
↪  lzComposeOpt)});
```

Alternative: if compose is explicitly out of scope and the team prefers to keep wTAO send-only, document that explicitly and consider rejecting non-empty `composeMsg` at the contract layer (e.g., set a `msgInspector` that reverts on any compose payload, or override `_buildMsgAndOptions` to revert when `_sendParam.composeMsg.length > 0`). That makes "compose unsupported" a clear contract-enforced contract rather than a quiet failure mode where users burn wTAO trying to use a feature that isn't wired up.

TAO: Acknowledged.

7.2.3 Foundry profile drift between production and CI - tests do not exercise the deployed bytecode

Description: `foundry.toml` defines two profiles with diverging optimization settings. `[profile.default]` (`via_ir=true`, `optimizer_runs=1000`) is what `make deploy-wtao` and `make deploy-wtao-direct` use to build the bytecode that ships to chain. `[profile.ci]` overrides to `via_ir=false`, `optimizer_runs=200`, and is what CI runs `forge test` / `forge coverage` under. `via_ir=true` and `via_ir=false` are different optimization passes that can produce materially different bytecode for the same source - codegen-sensitive issues (`via_ir`-specific compiler bugs, inlining differences, stack-scheduling edge cases) would not be surfaced by tests.

Consider dropping the `[profile.ci]` overrides so tests build the same bytecode that ships, OR add a second CI job that runs `forge test --profile default` alongside the existing CI run. The latter preserves CI speed under non-IR while ensuring production bytecode is also exercised.

TAO: Acknowledged.

7.3 Gas Optimization

7.3.1 Use low-level call to send ETH if return data is unimportant

Description: Use low-level call to send ETH if return data is unimportant in `wTAO::withdraw`:

```
-       (bool success,) = payable(msg.sender).call{value: wad}("");
+       bool success; assembly { success := call(gas(), caller(), wad, 0, 0, 0, 0)}
```

wTAO: Fixed in commit bc35833

Cyfrin: Verified.